

SDP



Copilot Context Engineering

Place Context, Inspect Delivery, Prove Compliance

Give every context rule an owner, an evidence path, and an approval boundary

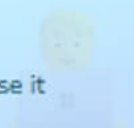
"Where should Copilot context live, and how do we prove it shaped the work?"

Placement assigns authority. Diagnostics reveal delivery.

Executable checks and owner approval prove compliance.

Developer

Place one rule where Copilot can use it



Team lead

Assign review and promotion authority



Platform engineer

Own shared context as a maintained product



Owner

Every shared rule names a producer and reviewer

Delivery

Loaded context shows eligibility, not obedience

Compliance

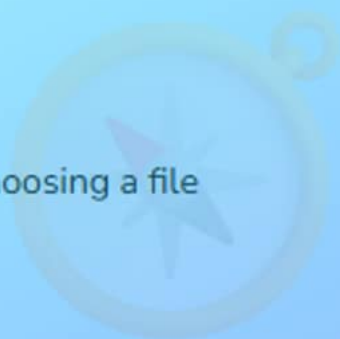
Tests, diff review, and approval close the claim

PART 1

Place Context

Owner and lifecycle

Classify the rule before choosing a file

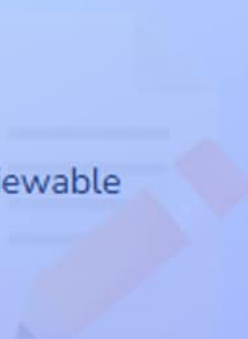


PART 2

Encode the Rule

Smallest fitting primitive

Make context bounded and reviewable

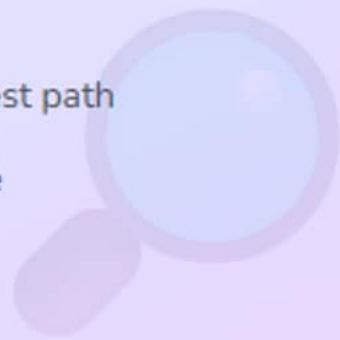


PART 3

Inspect Delivery

Loaded context and request path

Find the first broken stage



PART 4

Verify and Promote

Repair, test, and approve

Climb from eligibility to compliance



Click any section to jump directly there

Place Context by Owner and Lifecycle

Classify owner, lifetime, selector, and reviewer before choosing the storage surface.



Owner

Who may change the rule?



Lifetime

How long should it survive?



Reviewer

Who approves promotion?

Classification precedes configuration
↳ Owner + lifetime + selector + reviewer

Every Context Item Carries Four Design Decisions



Owner

Personal preference, repository policy, or organization control?



Lifetime

One request, several sessions, or the life of the codebase?



Selector

Everywhere, one path, an invoked task, or one agent role?



Evidence

Which signal proves loading, execution, and acceptance?

Shared Context Needs a Producer, Reviewer, and Consumer



Platform producer

A named maintainer owns conflicts, review cadence, version history, and retirement.



Code owner

Approves policy and public-contract changes through repository controls.



Contributor

May propose context and implementation changes, but cannot silently create team policy.

Encode the Smallest Primitive

Commit the narrowest artifact that carries the rule, selector, evidence, and authority boundary.



Artifact

Prefer a reviewable file



Selector

Bound when the rule applies



Authority

Separate tools from approval

Small surface, explicit contract
↳ Rule + scope + evidence + owner

Commit the Rule with Its Required Evidence

.github/copilot-instructions.md

Orders API Context

Owner: Orders API maintainers

Reviewer: Orders API code owner

Response Contract

- Return `Result<T, OrderError>`.
- Map errors to HTTP only in the route adapter.

Required Evidence

- `npm test -- orders`
- `npm run typecheck`

Bounded scope

Applies to Orders source and test work, not every repository task.

Separate authority

Tool access permits action; repository controls still decide merge approval.

Sensitive-data boundary

Credentials, customer records, payloads, and incident details stay out.

Promote Only When the Work Earns a Stronger Primitive

Owner, lifetime, and repetition determine the artifact

One task	Put explicit files and acceptance evidence in the request	Request
----------	---	---------

Personal	Keep a durable preference user-controlled and reviewable	Memory
----------	--	--------

Shared rule	Version a stable baseline with a named repository owner	Instruction
-------------	---	-------------

File pattern	Use path scope only when the pattern is stable and supported	applyTo
--------------	--	---------

Repeat task	Package a proven human-invoked sequence	Prompt/Skill
-------------	---	--------------

Bounded role	Assign a specialist tool boundary and explicit handoff	Agent
--------------	--	-------

Use one source of truth; duplication creates conflict without authority

PART 3

Inspect Context and Request Path

Trace eligibility, transport, and answer behavior without treating delivery as compliance.



Loaded

Was the file eligible?



Request Path

Did context travel?



Answer

Did output obey the rule?

Start at the first broken stage
↳ Delivery evidence ≠ compliance evidence

Name the Target, Boundary, and Acceptance Evidence

task request

Update `src/orders/update-order.ts` so a missing order returns the existing `OrderError.notFound` outcome.

Target

The decisive Orders file is explicit.

Boundary

The public response schema must remain stable.

Checks

Tests and type checking are acceptance evidence.

The Committed Selector Excludes the Orders Task

orders-context.broken.instructions.md

```
---
applyTo: src/payments/**/*.*ts
---

# Orders API Context
Return domain outcomes as Result<T, OrderError>.
Map HTTP responses only in the route adapter.
```

✅ Rule is valid

The domain and adapter guidance remains correct.

❌ Selector is wrong

The task targets `src/orders`, so this artifact is not eligible.

✚ Fixture is committed

The failure stays reproducible instead of becoming a story.

Illustrative Trace: Stop at the First Broken Stage

Illustrative diagnostic trace — exact labels vary by VS Code build

```
$ inspect loaded customizations
> "Target: src/orders/update-order.ts"
Illustrative diagnostic observation:
  Expected Orders instruction: absent
  Loaded selector: src/payments/**/*.*ts
  Request path: Orders file read and edited

✅ Diagnosis: valid rule, ineligible selector
  Generated output used an HTTP-shaped error in the domain layer
```

Illustrative trace · loading failed before compliance was tested

Loaded Context Proves Delivery, Not Obedience

Treat configuration delivery and output compliance as different claims



delivery evidence is not compliance evidence

Context Engineering evidence model

Loaded artifact

The client found and selected the instruction.

Request trace

The client assembled relevant context and tool activity.

Executable checks

Defined behavioral and type constraints hold.

Owner approval

The authorized reviewer accepts intent and compatibility.

 Inspect delivery to diagnose the path; inspect output and checks to judge compliance.

Transport Success and Session Continuity Need Separate Checks

🔌 MCP tool path

Server health proves only that a path exists

Tool success does not prove authoritative data

Inspect the invocation, result, and source authority

🧠 Context pressure

Capacity varies by model and session

After compaction, restate decisive constraints

Rerun acceptance checks instead of assuming continuity

PART 4

Repair Then Verify Compliance

Fix the selector, preserve the evidence, and promote only after executable checks and owner review.



Repair

Change the broken selector



Evidence

Climb all five rungs



Promote

Require owner approval

Observable proof before promotion

↳ Eligible → delivered → checked → approved

One Selector Change Restores Eligibility

✗ Broken fixture

Valid Orders rule

Payments path excludes the target

Expected instruction absent from loaded context

```
applyTo: src/payments/**/*.*ts
```

✓ Fixed fixture

Same Orders rule

Orders path includes the target

Instruction becomes eligible for the request

```
applyTo: src/orders/**/*.*ts
```

Illustrative Trace: Delivery Works After the Repair

Illustrative diagnostic trace — exact labels vary by VS Code build

```
$ inspect loaded customizations  
> "Target: src/orders/update-order.ts"  
Illustrative diagnostic observation:  
  orders-context.fixed.instructions.md: loaded  
  Target file: read and edited  
  Orders tests: exit 0  
  TypeScript check: exit 0  
  
✓ Delivery observed; continue to output and diff review
```

Illustrative trace · transport success still does not prove compliance

State Exactly What Each Signal Proves

No rung can make the claim assigned to the next



The north star is accepted delivery, not merely successful context transport

Executable Checks Test the Output, Not the Configuration

acceptance checks

```
npm test -- orders  
npm run typecheck  
git diff -- src/orders tests/orders
```

✓ Orders tests

Defined domain behavior remains correct.

◆ Type check

Changed TypeScript contracts remain coherent.

🔍 Diff review

HTTP mapping stays in the adapter and schema remains stable.

Promote Only Stable, Observable, Owned Context



Repeated value

Several bounded tasks show the rule is stable and useful.



Maintained product

A platform producer owns scope, conflicts, cadence, and retirement.



Authorized approval

The Orders API code owner approves policy and contract changes.

From Ambient Advice to Governed Context

Before

Rules live wherever someone remembers them

Selectors fail without a preserved fixture

Loaded context is mistaken for compliance

Shared guidance has no accountable producer

After

Every rule has an owner and lifecycle

Broken and fixed selectors stay committed

Delivery and compliance use distinct evidence

Platform producers maintain and retire shared rules

5 rungs

from eligible loading to owner approval

3 fixtures

promoted, broken, and repaired
evidence

1 owner

named for every shared context product

✓ What You Can Do Today

Today

- Classify one rule by owner and lifetime
- Name its delivery and compliance evidence
- Reject sensitive data from context artifacts

This Week

- Commit a scoped instruction and selector fixture
- Inspect loaded customization and request traces
- Add an executable check for the rule

This Month

- Assign platform producer ownership
- Define review cadence and retirement criteria
- Require code-owner approval for promotion

🔑 Key Takeaway

Treat context as a governed input whose delivery and compliance must be proven separately.

Official Documentation

[Custom instructions in VS Code](#)

Repository and path-scoped instruction behavior

[Repository custom instructions](#)

GitHub guidance for shared repository context

[Copilot Memory](#)

Current availability, scope, and management controls

[Chat Debug View](#)

Inspect request and diagnostic information

Diagnostics and Boundaries

[Troubleshoot AI in VS Code](#)

Current diagnostic surfaces and recovery guidance

[Use MCP servers in VS Code](#)

Server, discovery, invocation, and result boundaries

[Manage context for AI](#)

Context usage and compaction guidance



Place. Inspect. Prove.

Copilot Context Engineering

5 rungs

State exactly what each evidence signal proves

1 selector

Move a valid rule from ineligible to observable

3 fixtures

Preserve promoted, broken, and repaired states

Which rule in your repository needs an owner, selector, and compliance check?